

DRACARYS Platform - Technical Architecture & Stack

Date: September 25, 2026
Project: DRACARYS Web Platform

1. Complete Technology Stack

Frontend Architecture

- **Framework:** Next.js 15 (App Router)
- **UI Library:** React 19
- **Styling:** Tailwind CSS (Utility-first CSS)
- **Component Library:** shadcn/ui (Radix UI primitives)
- **Animations:** Framer Motion
- **Icons:** Lucide React & Custom SVGs

Backend & API

- **Runtime:** Node.js & Vercel Edge Network
- **API Paradigm:** Next.js Server Actions (RPC-style backend execution)
- **Validation:** Zod (Strict TypeScript schema validation)
- **Email Automation:** Resend (HTML template delivery)

Database & Data Modeling

- **Database:** PostgreSQL
- **ORM:** Prisma Client (Type-safe database querying)
- **Schema Architecture:** Relational schemas mapping Users, MemberProfiles, Projects, Applications, and Announcements.

Authentication & Security

- **Auth Provider:** Next-Auth v5 (Auth.js)
 - **Strategy:** JWT (JSON Web Tokens)
 - **Providers:** Google OAuth 2.0 & Credentials (bcryptjs hashed)
 - **Security Headers:** Helmet-style strict headers (CSP, HSTS, X-Frame-Options)
-

2. System Architecture

2.1. Rendering Strategy

DRACARYS utilizes a hybrid rendering approach:

- **React Server Components (RSC):** The vast majority of the application (including the public/team roster and dashboard overview) is rendered directly on the server. This results in zero client-side JavaScript bundling for these components, yielding lightning-fast SEO and page loads.
- **Client Components:** Interactivity (like the Onboarding Form, Framer Motion animations, and animated Admin buttons) is explicitly isolated using the "use client" directive to preserve performance.

2.2. Authentication Flow & Edge Middleware

The platform employs a robust edge-routing mechanism:

1. **Public Access:** Routes like /about, /team, and /founder bypass the middleware completely.
2. **JWT Sessions:** Upon Google OAuth login, Auth.js signs a secure JSON Web Token containing the user's id, role, and status.
3. **Edge Bouncer:** The Next.js middleware.ts runs on Vercel's Edge Network (closest to the user). It intercepts incoming requests to /dashboard. If the JWT is missing, it redirects to /login. If a non-admin attempts to access /dashboard/admin/*, the Edge network instantly blocks the request without ever spinning up the Node.js server.

2.3. Database Architecture

The PostgreSQL database is organized into distinct, normalized models:

- **User:** Handles core authentication, session management, and role-based access (SUPER_ADMIN, ADMIN, MEMBER).
- **MemberProfile:** A 1-to-1 relational extension of the User table storing specific DRACARYS metadata (Tags, Skills, Bio, LinkedIn, GitHub, Instagram).
- **JoinApplication & ProjectInquiry:** Isolates incoming requests from the public, maintaining an audit trail of approvals and rejections.

2.4. 5-Layer Defense-in-Depth Security

1. **Network Layer:** Vercel DDoS protection and strict Content-Security-Policy (CSP) blocking XSS and Clickjacking (frame-ancestors 'none').
2. **Edge Layer:** Middleware strictly routing based on encrypted JWT roles.
3. **API Layer:** Every Server Action rigorously validates session.user.id and role before executing.
4. **Database Layer:** Prisma ORM completely mitigates SQL injection via automated query parameterization.
5. **Form Validation:** Zod enforces strict validation criteria (e.g., URL formatting, string lengths) natively on the server to prevent malicious payload injection.